

1. Which code can be inserted to have the code print 2?

```
public class BirdSeed {  
    private int numberBags;  
    boolean call;  
  
    public BirdSeed() {  
        // LINE 1  
        call = false;  
        // LINE 2  
    }  
  
    public BirdSeed(int numberBags) {  
        this.numberBags = numberBags;  
    }  
  
    public static void main(String[] args) {  
        var seed = new BirdSeed();  
        System.out.print(seed.numberBags);  
    } }
```

- A. Replace line 1 with `BirdSeed(2);`.
- B. Replace line 2 with `BirdSeed(2);`.
- C. Replace line 1 with `new BirdSeed(2);`.
- D. Replace line 2 with `new BirdSeed(2);`.
- E. Replace line 1 with `this(2);`.
- F. Replace line 2 with `this(2);`.
- G. The code prints 2 without any changes.

Se debe asignar el valor que se desea imprimir antes del new, por lo que para llamar a un constructor dentro de otro constructor es importante hacerlo lo antes posible, lo insertamos en la línea 1 ya que debe ir antes de todo, sino Java mandaría un error de compilación

2. Which modifier pairs can be used together in a method declaration? (Choose all that apply.)

- A. `static` and `final`
- B. `private` and `static`
- C. `static` and `abstract`
- D. `private` and `abstract`
- E. `abstract` and `final`
- F. `private` and `final`

La opción A es valido ya que static asume que un método le pertenece a una clase y final redefine un método estático idéntico.

B es correcta ya que al ser private nadie puede acceder a el y static porque el método le pertenece a la clase y no a un objeto en específico

C es correcta ya que nadie puede acceder al método y final no puede ser sobrescrito por una subclase

Which of the following statements about methods are true? (Choose all that apply.)

- A. Overloaded methods must have the same signature.
- B. Overridden methods must have the same signature.
- C. Hidden methods must have the same signature.
- D. Overloaded methods must have the same return type.
- E. Overridden methods must have the same return type.
- F. Hidden methods must have the same return type.

Las opciones B y C son correctas los métodos de instancia y los estáticos deben tener la misma firma el nombre y los parámetros deben coincidir

4. What is the output of the following program?

```
1: class Mammal {
2:     private void sneeze() {}
3:     public Mammal(int age) {
4:         System.out.print("Mammal");
5:     } }
6: public class Platypus extends Mammal {
7:     int sneeze() { return 1; }
8:     public Platypus() {
9:         System.out.print("Platypus");
10:    }
11:    public static void main(String[] args) {
12:        new Mammal(5);
13:    } }
```

- A. Platypus
- B. Mammal
- C. PlatypusMammal
- D. MammalPlatypus
- E. The code will compile if line 7 is changed.
- F. The code will compile if line 9 is changed.

La clase padre no define un constructor sin argumentos por lo que en Platypus no manda llamar de forma explícita con un super(int) haciendo que el código no se ejecute

5. Which of the following complete the constructor so that this code prints out 50? (Choose all that apply.)

```
class Speedster {
    int numSpots;
}
public class Cheetah extends Speedster {
    int numSpots;

    public Cheetah(int numSpots) {
        // INSERT CODE HERE
    }

    public static void main(String[] args) {
        Speedster s = new Cheetah(50);
        System.out.print(s.numSpots);
    }
}
```

- A. `numSpots = numSpots;`
- B. `numSpots = this.numSpots;`
- C. `this.numSpots = numSpots;`
- D. `numSpots = super.numSpots;`
- E. `super.numSpots = numSpots;`
- F. The code does not compile regardless of the code inserted into the constructor.
- G. None of the above

numSpots debe ser llamado a la superclase para que el valor se quede grabado en la clase Speedster y pueda imprimir por lo que la opción E es correcta

6. Which of the following declare immutable classes? (Choose all that apply.)

```
public final class Moose {  
    private final int antlers;  
}
```

```
public class Caribou {  
    private int antlers = 10;  
}
```

```
public class Reindeer {  
    private final int antlers = 5;  
}
```

```
public final class Elk {}
```

```
public final class Deer {  
    private final Object o = new Object();  
}
```

- A. Moose
- B. Caribou
- C. Reindeer
- D. Elk**
- E. Deer**
- F. None of the above

D y E son correctas ya que es final y no puede heredarse, su constructor es private por lo que no se puede tener acceso, ambas cumplen con las características solicitadas

7. What is the output of the following code?

```
1: class Arthropod {
2:     protected void printName(long input) {
3:         System.out.print("Arthropod");
4:     }
5:     void printName(int input) {
6:         System.out.print("Spooky");
7:     } }
8: public class Spider extends Arthropod {
9:     protected void printName(int input) {
10:        System.out.print("Spider");
11:    }
12:    public static void main(String[] args) {
13:        Arthropod a = new Spider();
14:        a.printName((short)4);
15:        a.printName(4);
16:        a.printName(5L);
17:    } }
```

- A. SpiderSpiderArthropod
- B. SpidersSpidersSpider
- C. SpiderSpookyArthropod
- D. SpookySpiderArthropod
- E. The code will not compile because of line 5.
- F. The code will not compile because of line 9.
- G. None of the above

En la línea 14 el compilador busca un short como no encuentra ninguno lo promueve a un int imprimiendo Spider, en la línea 15 nuevamente busca un int por lo que imprime spider, por ultimo busca un Long, imprimiendo Arthropod por lo tanto A es correcta

8. What is the result of the following code?

```
1: abstract class Bird {  
2:     private final void fly() { System.out.println("Bird"); }  
3:     protected Bird() { System.out.print("Wow-"); }  
4: }  
5: public class Pelican extends Bird {  
6:     public Pelican() { System.out.print("Oh-"); }  
7:     protected void fly() { System.out.println("Pelican"); }  
8:     public static void main(String[] args) {  
9:         var chirp = new Pelican();  
10:        chirp.fly();  
11: } }
```

- A. Oh-Bird
- B. Oh-Pelican
- C. Wow-Oh-Bird
- D. Wow-Oh-Pelican**
- E. The code contains a compilation error.
- F. None of the above

Bird no se puede imprimir ya que es private final, esta protegido y no se puede heredar, por lo que la opción D es correcta

9. Which of the following statements about overridden methods are true? (Choose all that apply.)

- A. An overridden method must contain method parameters that are the same or covariant with the method parameters in the inherited method.
- B. An overridden method may declare a new exception, provided it is not checked.**
- C. An overridden method must be more accessible than the method in the parent class.
- D. An overridden method may declare a broader checked exception than the method in the parent class.
- E. If an inherited method returns void, then the overridden version of the method must return void.**
- F. None of the above

Void es covariante con void por lo que la opción E es correcta, y un método override no puede declarar nuevas excepciones por lo que la opción B es correcta

10. Which of the following pairs, when inserted into the blanks, allow the code to compile?
(Choose all that apply.)

```
1: public class Howler {  
2:     public Howler(long shadow) {  
3:         _____;  
4:     }  
5:     private Howler(int moon) {  
6:         super();  
7:     }  
8: }  
9: class Wolf extends Howler {  
10:     protected Wolf(String stars) {  
11:         super(2L);  
12:     }  
13:     public Wolf() {  
14:         _____;  
15:     }  
16: }
```

- A. `this(3)` at line 3, `this("")` at line 14
- B. `this()` at line 3, `super(1)` at line 14
- C. `this((short)1)` at line 3, `this(null)` at line 14
- D. `super()` at line 3, `super()` at line 14

En A `this(3)` llama a un constructor que acepte un `int`, en la línea 14 llama a un constructor que busque un `string`, como ambos parámetros cumplen el código compila, En C un `short` puede ser promovido a `int` por lo que cabe perfectamente, mientras que `null` al no tener un valor en específico puede ser asignado sin problema

11. What is the result of the following?

```
1: public class PolarBear {
2:     StringBuilder value = new StringBuilder("t");
3:     { value.append("a"); }
4:     { value.append("c"); }
5:     private PolarBear() {
6:         value.append("b");
7:     }
8:     public PolarBear(String s) {
9:         this();
10:        value.append(s);
11:    }
12:    public PolarBear(CharSequence p) {
13:        value.append(p);
14:    }
15:    public static void main(String[] args) {
16:        Object bear = new PolarBear();
17:        bear = new PolarBear("f");
18:        System.out.println(((PolarBear)bear).value);
19:    } }
```

- A. tacb
- B. tacf
- C. tacbf
- D. tcafb
- E. taftacb
- F. The code does not compile.
- G. An exception is thrown.

El valor inicial es tac, conforme el orden de compilación el stringBuilder siempre arranca con ese mismo valor, pasa por el constructor vacío y agrega b, teniendo tacb, el flujo regresa al constructor original y agrega f, asignando que s vale f

12. How many lines of the following program contain a compilation error?

```
1: public class Rodent {
2:     public Rodent(Integer x) {}
3:     protected static Integer chew() throws Exception {
4:         System.out.println("Rodent is chewing");
5:         return 1;
6:     }
```

```
7:  }  
8:  class Beaver extends Rodent {  
9:      public Number chew() throws RuntimeException {  
10:          System.out.println("Beaver is chewing on wood");  
11:          return 2;  
12:      } }
```

- A. None
- B. 1
- C. 2**
- D. 3
- E. 4
- F. 5

El error está en la línea 8 ya que no tiene un constructor super () por lo que es error de compilación, en la línea 9 debe ser static siguiendo las reglas de la herencia

13. Which of these classes compile and will include a default constructor created by the compiler? (Choose all that apply.)

A.

```
public class Bird {}
```

B.

```
public class Bird {  
    public bird() {}  
}
```

C.

```
public class Bird {  
    public bird(String name) {}  
}
```

D.

```
public class Bird {  
    public Bird() {}  
}
```

E.

```
public class Bird {  
    Bird(String name) {}  
}
```

F.

```
public class Bird {  
    private Bird(int age) {}  
}
```

G.

```
public class Bird {  
    public Bird bird() { return null; }  
}
```

La opción A es completamente valida, tanto en sintaxis como en lógica de constructores, la opción G tiene un método, no un constructor por lo que es bastante valido

14. Which of the following statements about inheritance are correct? (Choose all that apply.)

A. A class can directly extend any number of classes.

B. A class can implement any number of interfaces.

C. All variables inherit `java.lang.Object`.

D. If class A is extended by B, then B is a superclass of A.

E. If class C implements interface D, then C is a subtype of D.

F. Multiple inheritance is the property of a class to have multiple direct superclasses.

Una clase puede implementar múltiples interfaces por lo que B es correcta, una clase implementa una interfaz es un submúltiplo de dicha interfaz por lo que E es correcta, F describe perfectamente la herencia múltiple por lo que es correcta

15. Which statements about the following program are correct? (Choose all that apply.)

```
1: abstract class Nocturnal {  
2:     boolean isBlind();  
3: }  
4: public class Owl extends Nocturnal {  
5:     public boolean isBlind() { return false; }  
6:     public static void main(String[] args) {  
7:         var nocturnal = (Nocturnal)new Owl();  
8:         System.out.println(nocturnal.isBlind());  
9:     } }
```

- A. It compiles and prints true.
- B. It compiles and prints false.
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.
- G. None of the above

El método de la línea 2 no contiene el abstract por lo que no compilara haciendo la opción C correcta

16. What is the result of the following?

```
1: class Arachnid {  
2:     static StringBuilder sb = new StringBuilder();  
3:     { sb.append("c"); }  
4:     static  
5:     { sb.append("u"); }  
6:     { sb.append("r"); }  
7: }  
  
8: public class Scorpion extends Arachnid {  
9:     static  
10:    { sb.append("q"); }  
11:    { sb.append("m"); }  
12:    public static void main(String[] args) {  
13:        System.out.print(Scorpion.sb + " ");  
14:        System.out.print(Scorpion.sb + " ");  
15:        new Arachnid();  
16:        new Scorpion();  
17:        System.out.print(Scorpion.sb);  
18:    } }
```

- A. qu qu qumrcrc
- B. u u ucrcrm
- C. uq uq uqmcrcr
- D. uq uq uqrcrcrm**
- E. qu qu qumrcrc
- F. qu qu qucrorm
- G. The code does not compile.

sb queda como uq, después se generan los espacios imprimiendo el estado actual, se ejecutan los bloques de instancia en orden de aparición quedando como uqcr, pasando por los bloques de instancia del padre uqrcrcr, luego por el bloque de instancia de Scorpion uqrcrm por eso la opción D es correcta

17. Which of the following are true? (Choose all that apply.)

- A. `this()` can be called from anywhere in a constructor.
- B. `this()` can be called from anywhere in an instance method.
- C. `this.variableName` can be called from any instance method in the class.**
- D. `this.variableName` can be called from any static method in the class.
- E. You can call the default constructor written by the compiler using `this()`.
- F. You can access a private constructor with the `main()` method in the same class.**

C puede ejecutarse desde cualquier método de instancia pero no desde un método o inicializador estático, el método main se encuentra en la misma clase por lo que puede invocar métodos privados de dicha clase por eso F es correcta

18. Which statements about the following classes are correct? (Choose all that apply.)

```
1: public class Mammal {
2:     private void eat() {}
3:     protected static void drink() {}
4:     public Integer dance(String p) { return null; }
5: }
6: class Primate extends Mammal {
7:     public void eat(String p) {}
8: }
9: class Monkey extends Primate {

10:     public static void drink() throws RuntimeException {}
11:     public Number dance(CharSequence p) { return null; }
12:     public int eat(String p) {}
13: }
```

- A. The eat() method in Mammal is correctly overridden on line 7.
- B. The eat() method in Mammal is correctly overloaded on line 7.
- C. The drink() method in Mammal is correctly overridden on line 10.
- D. The drink() method in Mammal is correctly hidden on line 10.**
- E. The dance() method in Mammal is correctly overridden on line 11.
- F. The dance() method in Mammal is correctly overloaded on line 11.**
- G. The eat() method in Primate is correctly hidden on line 12.
- H. The eat() method in Primate is correctly overloaded on line 12.

Monkey lanza una nueva excepción que no es verificada por lo que esta permitida, el método dance esta sobrecargado porque las firmas no son iguales por eso D y F son correctas

19. What is the output of the following code?

```
1: class Reptile {  
2:     {System.out.print("A");}  
3:     public Reptile(int hatch) {}  
4:     void layEggs() {  
5:         System.out.print("Reptile");  
6:     } }  
7: public class Lizard extends Reptile {  
8:     static {System.out.print("B");}  
9:     public Lizard(int hatch) {}  
10:    public final void layEggs() {  
11:        System.out.print("Lizard");  
12:    }  
13:    public static void main(String[] args) {  
14:        var reptile = new Lizard(1);  
15:        reptile.layEggs();  
16:    } }
```

- A. AALizard
- B. BALizard
- C. BLizardA
- D. ALizard
- E. The code will not compile because of line 3.
- F. None of the above

El error ocurre en la línea 9 ya que al no escribir un constructor vacío Java hace uno en automático, por lo tanto en Reptile, Lizard busca un constructor sin argumentos, al no encontrarlo arroja un error

20. Which statement about the following program is correct?

```
1: class Bird {
2:     int feathers = 0;
3:     Bird(int x) { this.feathers = x; }
4:     Bird fly() {
5:         return new Bird(1);
6:     } }
7: class Parrot extends Bird {
8:     protected Parrot(int y) { super(y); }
9:     protected Parrot fly() {
10:         return new Parrot(2);
11:     } }
12: public class Macaw extends Parrot {
13:     public Macaw(int z) { super(z); }
14:     public Macaw fly() {
15:         return new Macaw(3);
16:     }
17:     public static void main(String... sing) {
18:         Bird p = new Macaw(4);
19:         System.out.print(((Parrot)p.fly()).feathers);
20:     } }
```

- A. One line contains a compiler error.
- B. Two lines contain compiler errors.
- C. Three lines contain compiler errors.
- D. The code compiles but throws a `ClassCastException` at runtime.
- E. The program compiles and prints 3.
- F. The program compiles and prints 0.

El código compila sin ningún problema, no existe error de sintaxis y la herencia esta aplicada correctamente, en la línea 19 imprime 3 ya que la variable `feathers` es visible por estar definida en la clase `Birds`

21. Which of the following are properties of immutable classes? (Choose all that apply.)

- A. The class can contain setter methods, provided they are marked `final`.
- B. The class must not be able to be extended outside the class declaration.
- C. The class may not contain any instance variables.
- D. The class must be marked `static`.
- E. The class may not contain any `static` variables.
- F. The class may only contain `private` constructors.
- G. The data for mutable instance variables may be read, provided they cannot be modified by the caller.

La opción B es correcta ya que para que una clase sea inmutable debe declararse como final, la opción C es correcta ya que se permite el acceso más no alterar su valor

22. What does the following program print?

```
1: class Person {
2:     static String name;
3:     void setName(String q) { name = q; } }
4: public class Child extends Person {
5:     static String name;
6:     void setName(String w) { name = w; }
7:     public static void main(String[] p) {
8:         final Child m = new Child();
9:         final Person t = m;
10:        m.name = "Elysia";
11:        t.name = "Sophia";
12:        m.setName("Webby");
13:        t.setName("Olivia");
14:        System.out.println(m.name + " " + t.name);
15:    } }
```

- A. Elysia Sophia
- B. Webby Olivia
- C. Olivia Olivia
- D. Olivia Sophia**
- E. The code does not compile.
- F. None of the above

La línea 11 utiliza la referencia person apuntando a sophia, las líneas 12 y 13 llaman al método sobrescrito estableciendo a Olivia en la línea 12, siendo la opción D correcta

23. What is the output of the following program?

```
1: class Canine {
2:     public Canine(boolean t) { logger.append("a"); }
3:     public Canine() { logger.append("q"); }
4:
5:     private StringBuilder logger = new StringBuilder();
6:     protected void print(String v) { logger.append(v); }
7:     protected String view() { return logger.toString(); }
8: }
9:
10: class Fox extends Canine {
11:     public Fox(long x) { print("p"); }
12:     public Fox(String name) {
13:         this(2);
14:         print("z");
15:     }
16: }
17:
18: public class Fennec extends Fox {
19:     public Fennec(int e) {
20:         super("tails");
21:         print("j");
22:     }
23:     public Fennec(short f) {
24:         super("eevee");
25:         print("m");
26:     }
27:
28:     public static void main(String... unused) {
29:         System.out.println(new Fennec(1).view());
30:     } }
```

A. qpz

B. qpzj

C. jzpa

D. apj

E. apjm

F. The code does not compile.

G. None of the above

Seguindo el orden de los constructores, obtenemos como primera impresión q, ejecutando la línea 11 siendo p el valor, luego la línea 14 dando z, por último la línea 21 añadiendo j a la cadena siendo B la opción correcta

24. What is printed by the following program?

```
1: class Antelope {
2:     public Antelope(int p) {
3:         System.out.print("4");
4:     }
5:     { System.out.print("2"); }
6:     static { System.out.print("1"); }
7: }
8: public class Gazelle extends Antelope {
9:     public Gazelle(int p) {
10:         super(6);
11:         System.out.print("3");
12:     }
13:
14:     public static void main(String hopping[]) {
15:         new Gazelle(0);
16:     }
17:     static { System.out.print("8"); }
18:     { System.out.print("9"); }
19: }
```

- A. 182640
- B. 182943
- C. 182493
- D. 421389
- E. The code does not compile.
- F. The output cannot be determined until runtime.

Primero imprime 1, seguido de 2, continuando con la secuencia de constructores, tomando primero 24 y luego 93 por lo que la opción C es correcta

25. Which of the following are true about a concrete class? (Choose all that apply.)

- A. A concrete class can be declared as `abstract`.
- B. A concrete class must implement all inherited abstract methods.
- C. A concrete class can be marked as `final`.
- D. A concrete class must be immutable.
- E. A concrete method that implements an abstract method must match the method declaration of the abstract method exactly.

Una clase concreta debe implementar todos los métodos abstractos por lo que B es correcta, las clases concretas pueden marcarse opcionalmente como final por eso C es correcta

26. What is the output of the following code?

```
4: public abstract class Whale {
5:     public abstract void dive();
6:     public static void main(String[] args) {
7:         Whale whale = new Orca();
8:         whale.dive(3);
9:     }
10: }
11: class Orca extends Whale {
12:     static public int MAX = 3;
13:     public void dive() {
14:         System.out.println("Orca diving");
15:     }
16:     public void dive(int... depth) {
17:         System.out.println("Orca diving deeper "+MAX);
18:     } }
```

- A. Orca diving
- B. Orca diving deeper 3
- C. The code will not compile because of line 4.
- D. The code will not compile because of line 8.**
- E. The code will not compile because of line 11.
- F. The code will not compile because of line 12.
- G. The code will not compile because of line 17.
- H. None of the above

Al realizar la conversión en la línea 7, esta permitido pero la referencia al método whale en la línea 8 ya no tiene acceso por lo que marca error de compilación